

```
// ===== СТРУКТУРА ПРОЕКТА =====
```

- data
- index.html
- stations.txt
- lib
- ESP32-vs1053_ext
- additional info
- Breadboard.jpg
- ESP32_dev_board.jpg
- MP3_Board.gif
- MP3_board_schematic.jpg
- Webstreams.txt
- examples
- vs1053_ext
- vs1053_ext.ino
- src
- vs1053b-patches-flac.h
- vs1053_ext.cpp
- vs1053_ext.h
- library.properties
- LICENSE
- README.md
- src
- audio_module.h
- config.h
- display_module.h
- main.cpp
- mqtt_module.h
- web_server_module.h
- platformio.ini

```
// ===== FILE: audio_module.h =====
```

```
#pragma once
#include <SPI.h>
#include "vs1053_ext.h"
#include "config.h"
```

```
VS1053* player = nullptr;
```

```
// Переменные для хранения значений ползунков веб-интерфейса
int currentVolume = 12; // Общая громкость (0-21)
int currentBass = 0; // Усиление баса (0-15 дБ) int currentTreble = 0; //
Усиление требла (-8...7 дБ)
```

```
void initAudio() {
    pinMode(VS1053_RST, OUTPUT);
    digitalWrite(VS1053_RST, LOW);
    delay(50);
    digitalWrite(VS1053_RST, HIGH);
    delay(50);
```

```
    player = new VS1053(VS1053_CS, VS1053_DCS, VS1053_DREQ, HSPI,
PIN_SPI_MOSI, PIN_SPI_MISO, PIN_SPI_SCK);
```

```
    player->begin();
```

```

player->setVolume(currentVolume);

// Передаем дефолтный тембр
uint8_t toneData[4] = {
  (uint8_t)currentTreble, // Усиление ВЧ (-8...7)
  6, // Частота среза ВЧ по умолчанию (6 кГц) (uint8_t)currentBass, //
Усиление НЧ (0...15)
  5 // Частота среза НЧ по умолчанию (50 Гц)
};
player->setTone(toneData);

Serial.println("[Звук]: Модуль VS1053 успешно запущен.");
}

void startRadioStream(const char* url) {
  if (player != nullptr) {
    player->connecttohost(url);
    player->setVolume(currentVolume);
  }
}

void updateVolume(int vol) {
  if (player != nullptr) {
    currentVolume = constrain(vol, 0, 21); player->setVolume(currentVolume);
    Serial.printf("[Звук]: Новая громкость: %d\n", currentVolume);
  }
}

// ИСПРАВЛЕНО: Упаковываем данные эквалайзера в массив из 4 байт по
правилам Wolle void updateTone(int bass, int treble) {
  if (player != nullptr) {
    currentBass = constrain(bass, 0, 15);
    currentTreble = constrain(treble, -8, 7);

    uint8_t toneData[4] = {
      (uint8_t)currentTreble, // Treble Gain
      6, // Treble Freq (6 kHz) (uint8_t)currentBass, // Bass Gain
      5 // Bass Freq (50 Hz)
    };

    player->setTone(toneData); // Передаем массив в метод setTone
    Serial.printf("[Звук]: Эквалайзер -> БАС: %d, ТРЕБЛ: %d\n", currentBass,
currentTreble); }
}

void loopAudio() {
  if (player != nullptr) { player->loop();
  }
}

// ===== FILE: config.h =====
#pragma once
#include <Arduino.h>
#include <LittleFS.h>

// Настройки Wi-Fi
const char* WIFI_SSID = "Sloboda100";

```

```

const char* WIFI_PASSWORD = "2716192023";

// Наша новая аппаратная схема пинов для Lolin S2 Mini
#define PIN_SPI_SCK 7
#define PIN_SPI_MISO 9
#define PIN_SPI_MOSI 11

#define VS1053_CS 12 // XCS
#define VS1053_DCS 14 // XD_CS
#define VS1053_DREQ 13 // DREQ
#define VS1053_RST 18 // XRESET

#define I2C_SDA 33
#define I2C_SCL 35

// Структура для динамического списка станций
struct Station {
    String name;
    String url;
};

// Максимально резервируем до 30 станций в памяти
const int MAX_STATIONS = 50;
extern Station stationList[MAX_STATIONS];
extern int stationCount;

// Переменные для хранения настроек звука
extern int currentVolume;
extern int currentBass;
extern int currentTreble;
// Функции для работы с файловой системой
void initFileSystem();
void loadPlaylist();

// --- НАСТРОЙКИ MQTT ДЛЯ ОФИЦИАЛЬНОЙ ИНТЕГРАЦИИ YORADIO ---
const char* MQTT_SERVER = "192.168.1.23"; // <-- IP-адрес вашего Home
Assistant const int MQTT_PORT = 1883;
const char* MQTT_USER = "";
const char* MQTT_PASSWORD = "";

// Жестко фиксируем базовый топик для интеграции автора yoradio
#define MQTT_CLIENT_ID "homeradio" // Имя вашего радио
#define MQTT_BASE_TOPIC "yoradio/homeradio"

// ===== FILE: display_module.h =====
#pragma once
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include "config.h"

Adafruit_SSD1306 display(128, 64, &Wire, -1);

void initDisplay() {

```

```

Wire.begin(I2C_SDA, I2C_SCL);
if(!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
Serial.println("✗ Ошибка: Дисплей SSD1306 не найден!");
}
display.clearDisplay();
display.setTextColor(SSD1306_WHITE); display.setTextSize(1);
display.setCursor(0,0);
display.println("Radio Loading...");
display.display();
}

void updateDisplay(const char* stationName, const char* status) {
display.clearDisplay();
display.setCursor(0, 0);
display.setTextSize(1);
display.println("--- INTERNET RADIO ---");
display.println("");
display.setTextSize(2);
display.println(stationName);
display.setTextSize(1);
display.println("");
display.print("Status: ");
display.println(status);
display.display();
}

```

```
// ===== FILE: main.cpp =====
```

```

#include <Arduino.h>
#include <WiFi.h>
#include <LittleFS.h>
#include <WebServer.h>
#include <esp_task_wdt.h> // <-- ПОДКЛЮЧИЛИ АППАРАТНЫЙ WATCHDOG
#include "config.h"
#include "display_module.h"
#include "audio_module.h"
#include "web_server_module.h"
#include "mqtt_module.h"

```

```

// Таймер для неблокирующей проверки связи Wi-Fi
unsigned long lastWiFiCheck = 0; const unsigned long wifiCheckInterval =
10000; // Проверяем сеть раз в 10 секунд

```

```

// Указываем компилятору, что эта функция находится в другом файле (в
модуле веб-сервера) extern void handleWebRequests();

```

```

// Создаем глобальные сетевые объекты для MQTT и Веб-сервера
WiFiClient espClient;
PubSubClient mqttClient(espClient);
WebServer server(80);
File uploadFile;

```

```

// Переменные состояния плейлиста и звука
Station stationList[MAX_STATIONS];
int stationCount = 0;
String currentTrack = "No Title";
String currentStationName = "No Station";

```

```

int currentStationIdx = -1;
bool changeStationFlag = false;
String customUrl = "";

// Функция для отдачи статических файлов (HTML/CSS/JS) из LittleFS
браузеру void handleStaticFile() {
    String path = server.uri();
    if (path.endsWith("/")) path += "index.html";

    String contentType = "text/html";
    if (path.endsWith(".css")) contentType = "text/css";
    else if (path.endsWith(".js")) contentType = "application/javascript";
    if (LittleFS.exists(path)) {
        File file = LittleFS.open(path, "r");
        server.streamFile(file, contentType);
        file.close();
    } else {
        server.send(404, "text/plain", "File Not Found в LittleFS");
    }
}

void initFileSystem() {
    if (!LittleFS.begin(true)) {
        Serial.println("[Файлы]: Ошибка LittleFS!");
    }
}

void loadPlaylist() {
    stationCount = 0;
    if (!LittleFS.exists("/stations.txt")) {
        Serial.println("[Файлы]: Файл stations.txt не найден.");
        return;
    }

    File file = LittleFS.open("/stations.txt", "r");
    if (!file) return;

    while (file.available() && stationCount < MAX_STATIONS) {
        String line = file.readStringUntil('\n');

        // Удаляем скрытые символы возврата каретки \r
        line.replace("\r", "");
        line.trim();

        if (line.length() < 5 || line.startsWith("#")) continue;

        int separatorIdx = line.indexOf(';');
        if (separatorIdx > 0) { stationList[stationCount].name = line.substring
(0, separatorIdx);

        String url = line.substring(separatorIdx + 1);
        // Если в конце строки случайно осталась точка с запятой, отрезаем её if
(url.endsWith(";")) {
            url = url.substring(0, url.length() - 1);
        }
        url.trim();
    }
}

```

```

stationList[stationCount].url = url;
stationCount++;
}
}
file.close();
Serial.printf("[Файлы]: Плейлист считан. Успешно добавлено станций: %d\n", stationCount); }

void setup() {
    Serial.begin(115200);

    // Инициализируем Аппаратный Watchdog на 8 секунд (перезагрузит чип при зависании)
    esp_task_wdt_init(8, true);
    esp_task_wdt_add(NULL); // Привязываем WDT к главному циклу loop()

    initDisplay();
    initFileSystem();
    loadPlaylist();
    initAudio();

    // НЕБЛОКИРУЮЩИЙ СТАРТ WI-FI (зашиваем автореконнект на уровне SDK)
    WiFi.mode(WIFI_MODE_STA); WiFi.setAutoReconnect(true);
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

    Serial.print("[Сеть]: Подключение к Wi-Fi...");
    unsigned long startAttempt = millis();

    // Ждем роутер не более 10 секунд. Если он не готов — идем дальше,
    // реконнект будет в loop while (WiFi.status() != WL_CONNECTED && millis() -
    // startAttempt < 10000) { delay(500);
    Serial.print(".");
    esp_task_wdt_reset(); // Продлеваем WDT во время старта
}

initWebServer();
initMQTT();

if (stationCount > 0) {
    currentStationIdx = 0;
    changeStationFlag = true;
}
}

void loop() {
    // СБРОС СТОРОЖЕВОГО ТАЙМЕРА (если loop выполняется — плата жива и не зависла)
    esp_task_wdt_reset();

    // Неблокирующая проверка связи по таймеру раз в 10 секунд
    if (millis() - lastWiFiCheck > wifiCheckInterval) {
        lastWiFiCheck = millis();
        if (WiFi.status() != WL_CONNECTED) {
            Serial.println("[Сеть]: Связь потеряна! Иницирую мягкое переподключение...");
            WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
        }
    }

    // Крутим сетевые сервисы только тогда, когда Wi-Fi реально подключен

```

```

if (WiFi.status() == WL_CONNECTED) {
    handleWebRequests();
    loopMQTT();
}

// Аудиопоток должен кормиться ВСЕГДА, даже если временно отвалилась сеть
loopAudio();

if (changeStationFlag) {
    changeStationFlag = false;
    if (currentStationIdx >= 0 && currentStationIdx < stationCount) {
        updateDisplay(stationList[currentStationIdx].name.c_str(),
            "Connecting..."); startRadioStream(stationList
            [currentStationIdx].url.c_str()); updateDisplay(stationList
            [currentStationIdx].name.c_str(), "PLAYING");

        // Записываем имя текущей станции, чтобы веб-интерфейс и MQTT сразу
        увидели её currentStationName = stationList[currentStationIdx].name;    }
    else if (customUrl.length() > 0) {
        updateDisplay("Custom URL", "Connecting..."); startRadioStream
        (customUrl.c_str());
        updateDisplay("Custom URL", "PLAYING");
        currentStationName = "Custom URL";
    }
}

// Колбэки Wolle для передачи логов
void vs1053_showstation(const char *info) {
    Serial.printf("[Библиотека]: Станция -> %s\n", info);
    currentStationName = String(info);
    mqttPublishTrack(info);
}

void vs1053_showstreamtitle(const char *info) {
    Serial.printf("[Библиотека]: Трек -> %s\n", info);
    currentTrack = String(info);
    mqttPublishTrack(info);

    if (stationCount > 0 && currentStationIdx >= 0) { updateDisplay
        (stationList[currentStationIdx].name.c_str(), info);
    } else {
        updateDisplay("Custom URL", info);
    }
}

// ===== FILE: mqtt_module.h =====
#pragma once
#include <WiFi.h>
#include <PubSubClient.h>
#include "config.h"
// Функции управления звуком из вашего audio_module
void updateVolume(int vol);
void updateTone(int bass, int treble);

// Подключаем объект плеера Wolle из вашего audio_module для остановки

```

```
потока #include "vs1053_ext.h"
extern VS1053* player;
```

```
// Даем доступ к объектам веб-клиента и MQTT (они созданы в main.cpp)
extern WiFiClient espClient;
extern PubSubClient mqttClient;
```

```
// Глобальные переменные состояния из main.cpp
extern int currentVolume;
extern int currentBass;
extern int currentTreble;
extern int currentStationIdx;
extern bool changeStationFlag;
extern String customUrl;
extern String currentStationName;
extern int stationCount;
extern Station stationList[MAX_STATIONS];
```

```
// Топики строго по реальному коду интеграции e2002/poradio
const char* topic_command = MQTT_BASE_TOPIC "/command"; // Сюда НА шлет
текстовые команды const char* topic_status = MQTT_BASE_TOPIC "/status"; //
Сюда радио шлет статус (JSON) const char* topic_playlist = MQTT_BASE_TOPIC
"/playlist"; // Сюда шлем ссылку на плейлист для НА const char*
topic_volume = MQTT_BASE_TOPIC "/volume"; // Сюда шлем сырую громкость для
volume_listener
```

```
String currentMqttTrack = "Radio Ready";
String currentMqttStatus = "playing"; // Может быть "playing" или
"stopped"
```

```
// Отправка ссылки на плейлист в НА
void mqttPublishPlaylist() {
    if (!mqttClient.connected()) return;
    String playlistUrl = "http://" + WiFi.localIP().toString() +
"/api/ha_playlist"; Serial.printf("[MQTT]: Публикация ссылки плейлиста
для НА -> %s\n", playlistUrl.c_str()); mqttClient.publish(topic_playlist,
playlistUrl.c_str(), true);
}
```

```
// Отправка статуса и громкости строго по структурам status_listener и
volume_listener void mqttPublishStatus() {
    if (!mqttClient.connected()) return;
```

```
    // 1. ОТПРАВЛЯЕМ ГРОМКОСТЬ В ОТДЕЛЬНЫЙ СЫРОЙ ТОПИКОМ (как требует
volume_listener) int haVolume = (int)((currentVolume * 254) / 21); if
(haVolume > 254) haVolume = 254;
    if (haVolume < 0) haVolume = 0;
    mqttClient.publish(topic_volume, String(haVolume).c_str(), true);
```

```
    // 2. ФОРМИРУЕМ JSON СТРОГО ПО КЛЮЧАМ СТРУКТУРЫ js['title'], js['name'],
js['on'], js['status'], js['station'] String track = currentMqttTrack;
    String station = currentStationName;
    track.replace("\\", "");
    station.replace("\\", "");
```

```
    if (station.length() == 0 || station == "No Station") station = "Sloboda
Radio"; if (track.length() == 0) track = "Radio Ready";
```

```

    int currentIdx = (currentStationIdx >= 0) ? currentStationIdx + 1 : 1;
    int isStatusPlaying = (currentMqttStatus == "playing") ? 1 : 0;

    String json = "{";
    json += "\"title\": \"" + track + "\","; // js['title']
    json += "\"name\": \"" + station + "\","; // js['name']
    json += "\"on\": 1,"; // js['on']
    json += "\"status\": " + String(isStatusPlaying) + ","; // js['status']
    json += "\"station\": " + String(currentIdx); // js['station']
    json += "}";

    mqttClient.publish(topic_status, json.c_str(), true);
}

// Разбор текстовых команд строго из методов set_command, set_volume,
set_source и кнопок НА void mqttCallback(char* topic, byte* payload,
unsigned int length) {
    String body = "";
    for (unsigned int i = 0; i < length; i++) body += (char)payload[i];
    body.trim();
    Serial.printf("[MQTT]: Получена текстовая команда -> %s\n", body.c_str());

    // Команда громкости от api.set_volume: "vol X"
    if (body.startsWith("vol ")) {
        int haVol = body.substring(4).toInt();
        int vsVol = (haVol * 21) / 254;
        updateVolume(vsVol);
    }
    // Команда выбора станции от api.set_source или списка: "play X"
    else if (body.startsWith("play ")) {
        int targetIdx = body.substring(5).toInt() - 1; // Вычитаем 1, так как у
автора список с 1 if (targetIdx >= 0 && targetIdx < stationCount) {
currentStationIdx = targetIdx;
        customUrl = "";
        changeStationFlag = true;
        currentMqttStatus = "playing";
    }
}
// Кнопка Вперед (async_media_next_track)
else if (body == "next") {
    if (stationCount > 0) {
        currentStationIdx = (currentStationIdx + 1) % stationCount; customUrl =
"";
        changeStationFlag = true;
        currentMqttStatus = "playing";
    }
}
// Кнопка Назад (async_media_previous_track)
else if (body == "prev") {
    if (stationCount > 0) {
        currentStationIdx = (currentStationIdx - 1 + stationCount) %
stationCount; customUrl = "";
        changeStationFlag = true;
        currentMqttStatus = "playing";
    }
}
}

```

```

// Кнопки Стоп / Пауза (async_media_stop, async_media_pause)
else if (body == "stop") {
    currentMqttStatus = "stopped"; changeStationFlag = false;
    if (player != nullptr) player->stop_mp3client();    Serial.println
("[MQTT]: Поток остановлен.");
}
// Кнопка ИГРАТЬ (async_media_play шлет строку "start")
else if (body == "start" || body == "play") {
    currentMqttStatus = "playing";
    changeStationFlag = true;
}
// Кнопка выключения питания (async_turn_off)
else if (body == "turnoff") {
    currentMqttStatus = "stopped";
    changeStationFlag = false;
    if (player != nullptr) player->stop_mp3client();
}

mqttPublishStatus();
}

void connectMQTT() {
    if (!mqttClient.connected()) {
        Serial.print("[MQTT]: Подключение к брокеру...");
        if (mqttClient.connect(MQTT_CLIENT_ID, MQTT_USER, MQTT_PASSWORD)) {
            Serial.println(" успешно!");
            mqttClient.subscribe(topic_command);
            mqttPublishPlaylist();
            mqttPublishStatus();
        } else { Serial.printf(" ошибка, rc=%d. Повтор через 5 сек.\n",
mqttClient.state()); }
    }
}

void initMQTT() {
    mqttClient.setServer(MQTT_SERVER, MQTT_PORT); mqttClient.setCallback
(mqttCallback);
}

void loopMQTT() {
    if (!mqttClient.connected()) {
        static unsigned long lastReconnect = 0;
        if (millis() - lastReconnect > 5000) {
            lastReconnect = millis();
            connectMQTT();
        }
    }
    mqttClient.loop();

    static unsigned long lastStatusTime = 0;
    if (millis() - lastStatusTime > 10000) {
        lastStatusTime = millis();
        mqttPublishStatus();
    }
}

void mqttPublishTrack(const char* trackInfo) {

```

```

currentMqttTrack = String(trackInfo);
currentMqttStatus = "playing";
mqttPublishStatus();
}

```

```

// ===== FILE: web_server_module.h =====
#include <WebServer.h>
#include <LittleFS.h>
#include "config.h"

```

```

// Даем доступ к глобальным переменным из main.cpp extern WebServer
server;
extern File uploadFile;
extern int currentVolume;
extern int currentBass;
extern int currentTreble;
extern int currentStationIdx;
extern bool changeStationFlag;
extern String customUrl;

```

```

// Подключаем наши строки состояния
extern String currentTrack;
extern String currentStationName;

```

```

// Прототипы функций
void loadPlaylist();
void handleStaticFile();
void updateVolume(int vol);
void updateTone(int bass, int treble);

```

```

void initWebServer() {
    // API получения расширенного статуса для ползунков и метаданных браузера
    радио server.on("/getstatus", HTTP_GET, []() {
        String track = currentTrack;
        String station = currentStationName;
        track.replace("\\", "\\");
        station.replace("\\", "\\");

        String json = "{\n";
        json += "  \"vol\": " + String(currentVolume) + ",\n";
        json += "  \"bass\": " + String(currentBass) + ",\n";
        json += "  \"treble\": " + String(currentTreble) + ",\n";
        json += "  \"track\": \"" + track + "\",\n";
        json += "  \"station\": \"" + station + "\"\n";
        json += "}";

        server.send(200, "application/json", json);
    });
}

```

```

// API отдачи текстового контента плейлиста в JS-парсер вашего браузера
радио (без изменений) server.on("/api/getplaylist", HTTP_GET, []() {
    if (LittleFS.exists("/stations.txt")) {
        File f = LittleFS.open("/stations.txt", "r"); server.streamFile(f,
"text/plain");
        f.close();
    } else {

```

```

server.send(200, "text/plain", "");
}
});

// ЭНДПОИНТ-КОНВЕРТЕР: Разделяет имя и ссылку знаком ТАБУЛЯЦИИ (\t)
строго под line.split('\t') в НА server.on("/api/ha_playlist", HTTP_GET,
[]()) {
    if (!LittleFS.exists("/stations.txt")) {
        server.send(200, "text/plain", "");
        return;
    }

    File f = LittleFS.open("/stations.txt", "r");
    String output = "";
    while (f.available()) {
        String line = f.readStringUntil('\n');
        line.replace("\r", "");
        line.trim();

        if (line.length() < 5 || line.startsWith("#")) continue;

        if (line.endsWith(";")) {
            line = line.substring(0, line.length() - 1);
        }

        int separatorIdx = line.indexOf(';');
        if (separatorIdx > 0) {
            String name = line.substring(0, separatorIdx);
            String url = line.substring(separatorIdx + 1);
            name.trim();
            url.trim();

            output += name + "\t" + url + "\n";
        }
    }
    f.close();
    server.send(200, "text/plain", output);
});

server.on("/api/vol", HTTP_GET, []()) {
    if (server.hasArg("val")) updateVolume(server.arg("val").toInt());
    server.send(200, "text/plain", "OK");
});

server.on("/api/tone", HTTP_GET, []()) {
    if (server.hasArg("bass") &&
        server.hasArg("treble")) {
        updateTone(server.arg("bass").toInt(),
            server.arg("treble").toInt());
    }
    server.send(200, "text/plain", "OK");
});

server.on("/api/set", HTTP_GET, []()) {
    if (server.hasArg("id")) {
        currentStationIdx = server.arg("id").toInt();
        customUrl = "";
        changeStationFlag = true;
    }
    server.send(200, "text/plain", "OK");
}

```

```

});

server.on("/api/custom", HTTP_GET, []() {
  if (server.hasArg("url")) {
    customUrl = server.arg("url");
    currentStationIdx = -1;
    changeStationFlag = true;
  }
  server.send(200, "text/plain", "OK");
});

// Прием файлов
server.on("/api/upload", HTTP_POST, []() {
  server.send(200, "text/plain", "OK");
}, []() {
  HTTPUpload& upload = server.upload();
  if (upload.status == UPLOAD_FILE_START) {  uploadFile = LittleFS.open
("/stations.txt", "w");
  } else if (upload.status == UPLOAD_FILE_WRITE) {
    if (uploadFile) uploadFile.write(upload.buf, upload.currentSize);  } else
if (upload.status == UPLOAD_FILE_END) {
  if (uploadFile) {
    uploadFile.close();
    Serial.println("[Файлы]: Большой плейлист успешно сохранен!");
  }
  loadPlaylist();
}
});

server.onNotFound(handleStaticFile);
server.begin();
Serial.println("[Веб]: Чистый REST API сервер успешно запущен.");
}

void handleWebRequests() {
  server.handleClient();
}

// ===== FILE: platformio.ini =====
[platformio]
data_dir = data

[env:lolin_s2_mini]
platform = espressif32@6.5.0
framework = arduino
board = lolin_s2_mini
board_build.f_cpu = 240000000L

; Включаем поддержку встроенного USB-порта для вывода в Serial Monitor
build_flags =
  -D MQTT_MAX_PACKET_SIZE=2048  -D CORE_DEBUG_LEVEL=4
  -D _DEBUG
  -D ARDUINO_USB_CDC_ON_BOOT=1
  --after no_reset ; <-- ДОБАВЬТЕ ЭТУ СТРОКУ (прикажет esptool не делать
ресет по USB)

```

```
board_build.filesystem = littlefs
```

```
lib_deps =  
adafruit/Adafruit GFX Library @ ^1.11.9  
adafruit/Adafruit BusIO @ ^1.14.5  
adafruit/Adafruit SSD1306 @ ^2.5.13  
knolleary/PubSubClient @ ^2.8
```